

LeetCode 2594: Minimum Time to Repair Cars

© Authored by Arpit Raj, LNMIIT Jaipur | July 11, 2026

Problem Statement & Core Intuition

We are given an array of mechanic ranks and an integer representing the number of cars that need to be repaired.

- A mechanic with rank r can repair n cars in $r \times n^2$ minutes.
- The goal is to determine the **minimum time** required to repair all the cars in the garage.
- **Crucial Observation:** All mechanics can work simultaneously. Thus, the total time required will be bounded by the maximum time taken by any single mechanic working in parallel to complete their assigned share of cars.

Example Walkthrough

Let's consider an example where $\text{ranks} = [4, 2, 3, 1]$ and $\text{cars} = 10$.

If we distribute the cars among the mechanics as follows:

- Mechanic 1 (rank 4) repairs 2 cars → $4 \times 2^2 = 16$ minutes
- Mechanic 2 (rank 2) repairs 2 cars → $2 \times 2^2 = 8$ minutes
- Mechanic 3 (rank 3) repairs 2 cars → $3 \times 2^2 = 12$ minutes
- Mechanic 4 (rank 1) repairs 4 cars → $1 \times 4^2 = 16$ minutes

Because they are working simultaneously, the overall time taken is simply the maximum of these individual durations, which results in **16 minutes**.

Mathematical Approach & Logic

Key Concept: If the mechanics can repair all cars within a time limit T , they can inherently do it in $T+1$, $T+2$, and so forth. This combination of seeking a "Minimum" bound over a "Monotonic" function strongly points toward using **BINARY SEARCH** on the answer space.

Deriving the Search Logic:

We are given the relationship: $Time = rank \times cars^2$

To find out how many cars a mechanic can repair within a given time limit, we rearrange the formula:

$$cars^2 = Time / rank$$

$$cars = \lfloor \sqrt{(Time / rank)} \rfloor$$

This derived formula yields the exact number of whole cars a mechanic of a specific rank can repair in a given time. During our binary search, for any mid-point time limit, we sum the cars each mechanic can repair. If this sum meets or exceeds the required number of cars, the time limit is feasible.

C++ Solution Code

```
class Solution {
public:

    bool check(vector<int>& ranks, int cars, long long time) {
        long long done = 0;
        for(int r : ranks) {
            done += sqrt(time / r);    //  $t=rn^2$ , so rearrange to get  $n = \text{root of } t/r$ 
        }

        if(done >= cars) {
            return true;
        }

        return false;
    }

    long long repairCars(vector<int>& ranks, int cars) {
        long long l = 1; //  $1*(1)^2$  (from constraints)
        long long mn = *min_element(ranks.begin(), ranks.end()); // fastest mechanic at
        long long h = mn * 1LL * cars * cars;
        long long ans = h;

        while(l <= h) {
            long long mid = l + (h - l) / 2;

            if(check(ranks, cars, mid)) {
                ans = mid;
                h = mid - 1;
            } else {
                l = mid + 1;
            }
        }

        return ans;
    }
};
```